StudyMate Project Report

1. Cover Page

Project Title: StudyMate
Student Name: [ADD NAME]
Roll Number: [ADD ROLL NUMBER]
Course/Class: [ADD COURSE]
College Name: [ADD COLLEGE NAME]
Academic Year: [ADD YEAR]

---

2. Certificate

This is to certify that **[ADD NAME]**, bearing Roll Number **[ADD ROLL NUMBER]**, of **[ADD COURSE]**, **[ADD COLLEGE NAME]**, has completed the project titled **"StudyMate"** during the academic year **[ADD YEAR]**.

The project report describes the application and implementation submitted for academic evaluation.

Project Guide	Head of Department
Signature: _____	Signature: _____
Date: _____	College Seal: _____

3. Declaration

I, **[ADD NAME]**, declare that this report titled **"StudyMate"** describes the project developed and submitted by me for academic purposes. The report is based on the source code, templates, database models, tests, and configuration present in the project. Any implementation not present in the project is identified as not implemented or as a possible future improvement.

Place: _____
Date: _____
Student Signature: _____

4. Acknowledgement

I would like to thank my project guide, faculty members, and **[ADD COLLEGE NAME]** for their guidance and support during the development of StudyMate. I also acknowledge the documentation and open-source tools provided by the Django, Python, Bootstrap, and other project technology communities.

5. Abstract

StudyMate is a web application developed with Python and Django to help an individual student organize academic subjects and related study tasks. A registered user can create subjects, add tasks with a deadline and priority, view task details, edit or delete records, and mark tasks as completed or pending. The authenticated dashboard summarizes subject and task counts, completion progress, upcoming tasks, recent tasks, and progress by subject. Users can also update basic profile information.

The application uses Django's built-in user authentication and administration facilities. Its two application data models are `Subject` and `StudyTask`, with records associated with a user. SQLite is the default database for local development. The settings can use PostgreSQL when a `DATABASE_URL` is supplied. The interface is rendered by Django templates and styled with Bootstrap 5, Bootstrap Icons, and project CSS. There is no separate REST API or mobile application in the current implementation.

6. Introduction

6.1 Background

Students often keep subject notes, assignments, and deadlines in separate places. This can make it harder to see what work is pending or what should be done next. StudyMate provides a single web interface for a student to keep basic subject and study-task records together.

6.2 Purpose of StudyMate

The purpose of StudyMate is to organize a user's subjects and study tasks and provide a simple view of task status and completion progress.

6.3 Need for the System

The project addresses a small, practical organization need: associating each task with a subject, recording a deadline and priority, and viewing pending and completed work in one application. It is a personal planning tool; collaboration, teacher accounts, and institutional workflows are not implemented.

7. Problem Statement

When academic subjects, task descriptions, deadlines, and completion status are recorded in separate notes or lists, a student may have to check several places to understand current work. StudyMate provides authenticated pages where these records can be created, viewed, updated, filtered, and tracked.

8. Objectives of the Project

- Provide user registration and login.
- Allow each signed-in user to maintain subject records.
- Allow users to associate study tasks with their own subjects.
- Record a task title, description, priority, deadline, and completion status.
- Provide search, filtering, and sorting for tasks.
- Summarize counts and completion percentages on a dashboard and profile page.
- Provide a browser-based interface that adapts to different screen widths.
- Support local SQLite and a configurable PostgreSQL database URL.

9. Scope of the Project

The current scope includes a public home page, student account registration and login, authenticated subject and task management, task filtering, a dashboard, basic profile editing, and Django's built-in admin site.

The application does not currently provide shared class groups, teacher or institution accounts, calendars or reminders, file uploads, email verification, password recovery pages, chat, mobile apps, or a public API. These should not be considered part of the delivered functionality.

10. Existing System

The existing approach considered here is manual organization using separate notes, documents, or lists. Such an approach can require repeated checking and does not automatically provide the task counts and completion summaries available in StudyMate. This description is general context for the problem and does not claim that a particular existing software product was evaluated.

11. Proposed System

StudyMate places subject and task records in one authenticated web application. A task belongs to a subject and user, and includes a deadline, priority, and completion flag. The task list supports title search, subject/priority/status filters, and sorting. The dashboard calculates summary values from the signed-in user's records and displays upcoming and recent tasks.

12. Feasibility Study

12.1 Technical Feasibility

The project uses Python and Django, with Django templates, static CSS/JavaScript, and a relational database. Its dependencies and deployment configuration are present in the repository. It was designed to run locally with SQLite and can be configured for PostgreSQL through `DATABASE_URL`. Technical feasibility is suitable for a small student project. A production deployment should use persistent database storage.

12.2 Economic Feasibility

The application is built with open-source software. Local development does not require a paid hosting service. Cloud hosting and managed database free tiers are subject to provider availability, limits, and terms; ongoing free hosting is not guaranteed by the source code.

12.3 Operational Feasibility

The interface is accessed through a web browser. A student registers, signs in, and uses the navigation links to manage their own subjects, tasks, and profile. Django admin is available to authorized staff users for administrative record management.

13. Technologies Used

Technology	Purpose	Where it is used in StudyMate
Python 3.12 (project deployment configuration specifies 3.12.4 for Render)	Backend programming language	Django project, models, forms, views, tests, and management command
Django 6.1.1	Web framework, ORM, templates, forms, routing, authentication, and admin	`manage.py`, `studymate/`, and `planner/`
SQLite	Default relational database	`db.sqlite3` for local/default configuration
PostgreSQL	Optional relational database for deployment	Selected through `DATABASE_URL`; driver supplied by `psycopg2-binary`
HTML and Django Template Language	Server-rendered page structure and dynamic content	`templates/`
CSS	Application-specific presentation	`static/css/style.css`
JavaScript (vanilla)	Small client-side interactions	`static/js/main.js`
Bootstrap 5.3.3	Responsive layout and common UI components	Loaded from jsDelivr CDN in `templates/base.html`
Bootstrap Icons 1.11.3	Interface icons	Loaded from jsDelivr CDN in `templates/base.html`
WhiteNoise 6.12.0	Static-file middleware/storage support	Django middleware and static-file settings
dj-database-url 3.1.2	Parse `DATABASE_URL` into Django database settings	`studymate/settings.py`
psycopg2-binary 2.9.13	PostgreSQL database driver	Optional PostgreSQL connections

Technology	Purpose	Where it is used in StudyMate
Gunicorn 26.2.0	WSGI server for the Render deployment configuration	`render.yaml` start command
Vercel	Serverless Django hosting configuration	`vercel.json`; Vercel detects the Django project from `manage.py`
Git and GitHub	Source control and hosted source repository	Project source management and deployment integration

The dependency versions above are read from `requirements.txt`. No React, Node.js frontend, or separate API framework is present.

14. System Requirements

14.1 Hardware Requirements

The project does not define formal minimum hardware specifications. Development requires a computer capable of running the selected Python environment and a modern web browser. Internet access is needed for hosted access and for the Bootstrap assets referenced from the CDN.

14.2 Software Requirements

Software	Requirement or note
Operating system	The project can run in a Python environment; development was performed on Windows.
Python	Local environment reported as Python 3.12.4; Render configuration specifies Python 3.12.4.
Django	6.1.1, as pinned in `requirements.txt`.
Browser	A current browser with HTML, CSS, and JavaScript support.
Dependencies	Install with `pip install -r requirements.txt`.
Database	SQLite by default; PostgreSQL when a valid `DATABASE_URL` is configured.

Software	Requirement or note
Optional hosting tools	Vercel configuration is present; a Render blueprint and build script are also present.

15. System Architecture

StudyMate uses Django's server-rendered Model–Template–View (MTV) pattern. The browser sends an HTTP request to Django's URL configuration. A view performs authentication checks and database queries or form processing through Django's ORM. The view returns an HTML template response or redirects the browser. CSS and JavaScript are served as static files.

```
Web browser
    |
    | HTTP request / HTML form submission
    v
Django URL configuration (studymate/urls.py, planner/urls.py)
    |
    v
Views and authentication (planner/views.py, Django auth)
    |                          |
    | ORM queries/forms         | HTML response
    v                          v
SQLite or PostgreSQL       Django templates
                               |
                               v
                          Browser renders HTML

Static CSS/JavaScript: static/css/style.css, static/js/main.js
```

There is no separate frontend service or REST API in the source. On Vercel, the project configuration asks Django to run database migrations during the build. On Render, `render.yaml` describes a Gunicorn web service and a PostgreSQL database.

16. System Modules

Module	Purpose	Main functionality
Public home	Introduce the application	Displays StudyMate's purpose, feature summaries, workflow explanation, and links to register or log in.
Registration and authentication	Create and access user accounts	Registration checks duplicate username/email and matching passwords. Django's built-in login and logout views manage sessions.

Module	Purpose	Main functionality
Dashboard	Summarize one user's study records	Shows subject/task counts, completed and pending counts, overall completion percentage, upcoming tasks, recent tasks, and per-subject progress.
Subject management	Organize academic subjects	List, add, edit, and delete the signed-in user's subjects; show subject description, creation date, and task count.
Task management	Record and maintain study work	Add, view, edit, and delete tasks; associate a task with a subject; store priority, deadline, description, and status; mark tasks completed or pending.
Task search and filters	Find relevant tasks	Search title text; filter by subject, priority, and completion status; sort by deadline, priority, or creation date.
Profile	View and edit basic account details	Displays username, email, subject/task counts, completion count, and progress; updates first name, last name, and email.
Django administration	Administrative record management	Registers `Subject` and `StudyTask` in Django admin with list display, filters, ordering, and search fields. Requires an appropriately authorized Django account.
Sample data command	Create demonstration records	`python manage.py seed_data` creates or updates a demonstration student, sample subjects, and sample tasks. It is optional and is not an automatic production initialization step.

The only implemented application-specific modules in the Python app are models, forms, views, URL routes, admin configuration, tests, migrations, and the sample-data management command. Separate payment, messaging, notification, analytics, or API modules are not implemented.

17. Database Design

17.1 Database Name and Backend

The default database is the SQLite file `db.sqlite3` in the project directory. `studymate/settings.py` uses this default unless `DATABASE_URL` is supplied, in which case `dj-database-url` configures the specified database. PostgreSQL support is included in the dependencies. The deployed Vercel environment previously configured without `DATABASE_URL` uses the SQLite fallback; SQLite files on a serverless host are not reliable persistent storage.

17.2 Application Tables

The initial migration creates the following application tables (Django derives their default table names from the app and model names):

Table	Important fields	Purpose
`planner_subject`	`id` (primary key), `user_id` (foreign key), `name` (up to 100 characters), `description` (optional text), `created_at`	Stores a subject owned by a Django user. Subject records are ordered by name.
`planner_studytask`	`id` (primary key), `user_id` (foreign key), `subject_id` (foreign key), `title` (up to 200 characters), `description` (optional text), `priority` (`LOW`, `MEDIUM`, or `HIGH`), `deadline`, `completed`, `created_at`, `updated_at`	Stores a task, its owner, subject, scheduling and priority information, and completion state. Tasks are ordered by deadline and creation time.
`auth_user`	Django user fields including `id`, `username`, `password` hash, `email`, `first_name`, `last_name`, `is_active`, `is_staff`, and `is_superuser`	Stores accounts using Django's built-in user model. StudyMate does not define a custom user model.

Django migrations also create framework tables used for permissions, content types, migrations, sessions, and admin logging as applicable. Exact system-table details are managed by the installed Django version.

17.3 Relationships

- One Django user can own multiple subjects.
- One Django user can own multiple study tasks.
- One subject can have multiple tasks.
- Each subject and task belongs to one user.
- Deleting a user cascades to that user's subjects and tasks. Deleting a subject cascades to its related tasks.
- The task stores both `user_id` and `subject_id`; views and the task form limit operations and subject choices to the signed-in user's records.

18. System Design

18.1 Use Case Description

Actor	Use case	Result
Visitor	Open home page	Sees an overview and registration/login links.
Visitor	Register	Creates a user account after form validation, then is directed to login.

Actor	Use case	Result
Registered user	Log in / log out	Starts or ends a Django authentication session.
Registered user	Manage subjects	Creates, views, updates, and deletes their subjects.
Registered user	Manage study tasks	Creates, views, updates, deletes, completes, or reopens their own tasks.
Registered user	Find tasks	Searches and applies subject, priority, status, and sort options.
Registered user	Review progress	Views summary statistics on the dashboard and profile.
Django staff/admin user	Use Django admin	Manages registered application records subject to Django permissions.

There are no implemented teacher, class, or group actors and no sharing workflow between users.

18.2 Data Flow

1. A user opens a page or submits a form in the browser. 2. Django matches the request path to a named route. 3. The selected view checks authentication where required. 4. The view reads or updates records through Django models and validates submitted form data. 5. The view renders a template or redirects to another route. 6. The browser displays the returned page. For task filtering, query-string values are read from the GET request.

18.3 Entity–Relationship Description

```
auth_user 1 -------- many planner_subject
auth_user 1 -------- many planner_studytask
planner_subject 1 -- many planner_studytask
```

`planner_studytask` has foreign keys to both `auth_user` and `planner_subject`.

18.4 User Workflow

```
Home -> Register -> Login -> Dashboard
                         |       |
                         |       +-> Subjects -> Add/Edit/Delete
                         |
                         +----------> Tasks -> Add/View/Edit/Delete
                                      |        -> Filter/Sort/Search
                                      |        -> Mark Complete/Pending
                         +----------> Profile -> Update account details
```

19. Implementation

File or folder	Purpose
`manage.py`	Django command-line entry point.
`studymate/settings.py`	Installed apps, middleware, templates, database, static files, security, and environment-based deployment settings.
`studymate/urls.py`	Root routes for planner pages, login, logout, admin, and custom error handlers.
`studymate/wsgi.py`, `studymate/asgi.py`	WSGI and ASGI application entry points.
`planner/models.py`	Defines `Subject` and `StudyTask`.
`planner/forms.py`	Registration, subject, task, and profile forms; task choices are restricted to the supplied user's subjects.
`planner/views.py`	Registration, dashboard, subject/task CRUD, task list filters, profile update, and custom error responses.
`planner/urls.py`	Named application routes for home, registration, dashboard, subjects, tasks, and profile.
`planner/admin.py`	Admin list displays, search, filters, and ordering for subjects and tasks.
`planner/migrations/`	Database schema migration for the application models.
`planner/tests.py`	Automated tests for home, registration/login, authentication, subject operations, task operations, and user isolation.
`planner/management/commands/seed_data.py`	Optional command to create demonstration data.

File or folder	Purpose
`templates/`	HTML templates for public, account, dashboard, subject, task, profile, and error pages.
`static/css/style.css`	Application-specific styling and responsive adjustments.
`static/js/main.js`	Confirmation dialogs for links marked for confirmation, auto-dismissal of alerts, and password visibility toggles.
`requirements.txt`	Pinned Python dependencies.
`.env.example`	Names and placeholder examples for environment configuration; it is not a real credential file.
`vercel.json`	Vercel build configuration; runs Django migrations. Vercel's Django integration detects the project from `manage.py`.
`render.yaml`, `build.sh`	Alternative Render web-service/database blueprint and build steps.
`.gitignore`	Excludes environment secrets, SQLite files, virtual environments, bytecode, and generated static files.

The registration form checks username and email uniqueness and verifies that the two submitted password values match. The view creates users through Django's user manager. The dashboard calculates counts and percentages from the current user's records; it does not store a separate progress score.

20. User Interface

The pages below exist as templates. The bracketed lines are locations to insert screenshots captured from the running application; they are not screenshots embedded in this report.

20.1 Public Home Page

Displays the “Plan. Study. Achieve.” introduction, feature summaries, a short three-step workflow, benefits, and registration/login or dashboard links depending on authentication state.

[Insert Screenshot: Home Page]

20.2 Registration Page

Provides username, email, password, and confirm-password fields. Form errors appear on the page.

[Insert Screenshot: Registration Page]

20.3 Login Page

Provides username and password inputs, an option to show or hide the password, and a link to registration.

[Insert Screenshot: Login Page]

20.4 Dashboard

Displays counts for subjects and tasks, completed and pending counts, an overall progress bar, upcoming tasks, recent tasks, and progress summaries by subject.

[Insert Screenshot: Dashboard]

20.5 Subjects

The subject list displays each subject's name, description, task count, creation date, and edit/delete actions. Add and update share a subject form.

[Insert Screenshot: Subjects Page]

[Insert Screenshot: Add or Edit Subject Page]

20.6 Tasks

The task list displays subject, priority, deadline, completion status, and actions. Search and filter controls are available for title, subject, priority, status, and sort order. The task form supports add/edit; the detail page shows task information and status actions.

[Insert Screenshot: Tasks Page]

[Insert Screenshot: Add or Edit Task Page]

[Insert Screenshot: Task Details Page]

20.7 Profile

Displays the user's username, email, totals, completed-task count, progress percentage, and a form to update first name, last name, and email.

[Insert Screenshot: Profile Page]

20.8 Error Pages and Administration

Custom templates exist for 404 and 500 responses. The Django admin login and admin pages are provided by Django and are not custom StudyMate templates.

[Insert Screenshot: 404 Page, if required]

[Insert Screenshot: Django Admin Login]

21. Testing

The repository contains ten Django automated test methods. The project test suite was run locally in development mode and reported ten passing tests. The table reflects those automated tests; it does not claim that every browser workflow or cloud-hosted database behavior has been tested.

Test Case ID	Feature	Test Scenario	Expected Result	Actual Result	Status
TC-01	Home	Request the named home route	HTTP 200 and page contains StudyMate	HTTP 200; StudyMate was present	Passed
TC-02	Registration	Submit valid unique user details with matching passwords	Redirect to login and create user	Redirected to login; user existed	Passed
TC-03	Login	Submit valid credentials	Redirect to dashboard	Redirected to dashboard	Passed
TC-04	Authentication	Request dashboard while logged out	Redirect to login	Redirected to login	Passed
TC-05	Subject creation	Submit a subject form while signed in	Save subject for current user and redirect to list	Subject was created; redirected to list	Passed
TC-06	Subject editing	Update a subject owned by current user	Save changes and redirect to list	Subject name was updated; redirected to list	Passed
TC-07	Subject deletion	Delete a subject owned by current user	Remove subject and redirect to list	Subject was removed; redirected to list	Passed
TC-08	Task creation	Submit a task with valid subject, priority, and deadline	Save task for current user and redirect to tasks	Task was created; redirected to task list	Passed
TC-09	Task completion and deletion	Mark a task complete, then delete it	Completion flag changes; task is removed when deleted	Both operations were verified	Passed
TC-10	User data isolation	Try to edit another user's subject	Request returns not found	HTTP 404 was returned	Passed

Automated tests for task search/filter combinations, task edit/pending actions, profile validation, admin permissions, PostgreSQL deployment, and the live Vercel runtime are not present in `planner/tests.py`.

22. Security Features

Implemented security-related controls include:

- Django's built-in authentication and session mechanisms.
- Login-required decorators for dashboard, subject, task, and profile views.
- Querysets constrain subject/task reads and changes to the signed-in user; a test verifies cross-user subject access returns 404.
- CSRF middleware is enabled, and rendered POST forms include Django CSRF tokens.
- Registration checks for duplicate usernames and email addresses and checks that password and confirmation match.
- Django's standard password hash handling is used when creating users through the user manager.
- The project has default Django password-validator settings, but the custom registration form does not explicitly call Django's password validation function. Those validators should therefore not be assumed to enforce password policy at signup.
- Production settings read `SECRET_KEY`, `DEBUG`, allowed hosts, and CSRF trusted origins from environment variables. With `DEBUG=False`, the known local fallback secret is rejected.
- HTTPS-related settings enable SSL redirect, HSTS, and secure session/CSRF cookies when debug mode is off. A forwarded-protocol header is configured for proxy hosting.
- `.gitignore` excludes `.env`, SQLite files, virtual environments, and generated static files.

Security limitations visible in the implementation include delete and task status-changing actions being linked through GET routes rather than POST-only actions. Email verification and password reset are not implemented. The optional sample-data command creates a predictable demonstration account and should not be run as production account provisioning.

23. Advantages

- Keeps a user's subjects and their study tasks in one interface.
- Associates tasks with a subject, deadline, priority, and completion state.
- Provides practical task search, filters, and sorting.
- Summarizes progress without requiring users to calculate counts manually.
- Uses Django's built-in authentication, ORM, template engine, and admin.
- Provides per-user filtering in application views.
- Uses responsive Bootstrap layout and includes custom CSS for page styling.
- Supports SQLite for local development and has configuration for PostgreSQL.

24. Limitations

- StudyMate is a personal planning application; there are no collaboration, teacher, or class management features.
- No REST API or mobile application is implemented.
- Tasks have a date deadline but no time-of-day, recurring schedule, reminder, or calendar integration.
- No attachments, file storage, email verification, password recovery, or email notifications are implemented. The configured email backend is the console backend.
- The project has no custom profile model; profile information uses fields on Django's built-in user.
- Automated test coverage is limited to the ten scenarios listed above.
- The registration form does not explicitly invoke Django's configured password validators.

- Delete and task status actions use GET routes.
- SQLite is the default when no `DATABASE_URL` is supplied. SQLite on serverless hosting may be non-persistent; PostgreSQL is recommended for durable hosted user data.
- Bootstrap and Bootstrap Icons are loaded from a CDN, so those assets depend on external network availability in the browser.
- A sample-data management command includes a predictable demo account; it is for demonstration and is not suitable as production account creation.

25. Future Scope

The following are possible improvements only; they are **not currently implemented**:

- Use POST forms with CSRF protection for deletion and task status changes.
- Apply Django password validation explicitly during registration and add password reset and email verification workflows.
- Connect a persistent PostgreSQL database for production and document backup/restore procedures.
- Add automated tests for task filters, sorting, pending status, profile changes, and admin access.
- Add optional reminders, calendar export, and recurring tasks.
- Add task attachments with controlled file storage.
- Add teacher/class roles or shared planning only if the project requirements are expanded.
- Add a documented REST API if integration with a separate client becomes necessary.
- Host Bootstrap assets locally if CDN-independent rendering is required.

26. Conclusion

StudyMate is a Django web application for personal organization of academic subjects and study tasks. It implements account registration and login, subject and task management, task search and filters, completion tracking, a dashboard, profile editing, and Django admin integration. Its relational data model associates each record with a user and each task with a subject. The project is suitable as a small college web application and has local testing and deployment configuration. Persistent production use should be paired with PostgreSQL rather than relying on SQLite on serverless hosting.

27. References

The project uses or configures the following technologies and resources:

1. Python documentation — https://docs.python.org/3/ 2. Django documentation — https://docs.djangoproject.com/ 3. Django authentication, forms, ORM, templates, migrations, and admin documentation — https://docs.djangoproject.com/en/6.1/ 4. Django deployment and WSGI documentation — https://docs.djangoproject.com/en/6.1/howto/deployment/ 5. Bootstrap 5.3 documentation — https://getbootstrap.com/docs/5.3/ 6. Bootstrap Icons — https://icons.getbootstrap.com/ 7. WhiteNoise documentation — https://whitenoise.readthedocs.io/ 8. dj-database-url package — https://github.com/jazzband/dj-database-url 9. PostgreSQL documentation — https://www.postgresql.org/docs/ 10. psycopg2 documentation — https://www.psycopg.org/docs/ 11. Gunicorn documentation — https://docs.gunicorn.org/ 12. Vercel Django framework documentation — https://vercel.com/docs/frameworks/full-stack/django 13. Render documentation — https://render.com/docs/ 14. Project source files, migrations, templates, tests, `requirements.txt`, `render.yaml`, and `vercel.json`.